JavaScript

Interview Questions & Answers

Complete Reference: 88+ Questions Covering All Core Topics

Topics: Core JavaScript Fundamentals, Functions, Scope and this, Objects and Prototypes, Arrays, Asynchronous JavaScript, ES6+ Features, Classes and OOP, Error Handling, Modules, DOM and Browser APIs, Design Patterns and Best Practices, Memory Management and Performance, Advanced Concepts, Miscellaneous / Common Interview Questions, Security

From Fundamentals to Advanced Concepts

Table of Contents

Core JavaScript Fundamentals

Q1

What is JavaScript?

JavaScript is a lightweight, interpreted, object-oriented programming language with first-class functions. It is most well-known as the scripting language for Web pages, but it is also used in many non-browser environments such as Node.js. JavaScript is a prototype-based, multi-paradigm, single-threaded, dynamic language, supporting object-oriented, imperative, and declarative programming styles.

Q2

What are the data types in JavaScript?

JavaScript has 8 data types: Primitive Types (7): 1. String – e.g., "hello" 2. Number – e.g., 42, 3.14, NaN, Infinity 3. BigInt – e.g., 9007199254740991n 4. Boolean – true or false 5. undefined – variable declared but not assigned 6. null – intentional absence of value

```
7. Symbol        – unique identifier (ES6+)
```

Non-Primitive (Reference) Type (1):

```
8. Object        – includes arrays, functions, dates, etc.
```

typeof null returns "object" – this is a long-standing bug in JavaScript.

Q3

What is the difference between var, let, and const?

var:

```
- Function-scoped (or globally scoped if declared outside a function)
```

- Can be re-declared and updated - Hoisted and initialized to undefined let: - Block-scoped - Cannot be re-declared in the same scope - Can be updated

```
- Hoisted but NOT initialized (Temporal Dead Zone)
```

const: - Block-scoped - Cannot be re-declared or re-assigned - Must be initialized at declaration

```
- Hoisted but NOT initialized (Temporal Dead Zone)
  - Objects/arrays declared with const can still be mutated
```

Example:

```
var x = 1; var x = 2;   // OK
  let y = 1; let y = 2;   // SyntaxError
  const z = 1; z = 2;     // TypeError
```

Q4

What is hoisting in JavaScript?

Hoisting is JavaScript's default behaviour of moving declarations to the top of the current scope before code execution. Only declarations are hoisted, not initialisations. var hoisting: var declarations are hoisted and initialised to undefined.

```
console.log(a); // undefined
  var a = 5;
```

let/const hoisting: Hoisted but NOT initialised – accessing them before declaration throws a ReferenceError (Temporal Dead Zone).

```
console.log(b); // ReferenceError
  let b = 10;
```

Function declarations are fully hoisted (both name and body):

```
greet(); // "Hello"
  function greet() { console.log("Hello"); }
```

Function expressions are NOT fully hoisted:

```
sayHi(); // TypeError: sayHi is not a function
  var sayHi = function() { console.log("Hi"); };
```

Q5

What is the Temporal Dead Zone (TDZ)?

The Temporal Dead Zone is the period between entering a block scope and the point where a let or const variable is declared. During the TDZ, the variable exists in the scope but cannot be accessed – any attempt to read or write it throws a ReferenceError. Example:

```
{
    // TDZ for x starts here
    console.log(x); // ReferenceError: Cannot access 'x' before initialization
    let x = 5;      // TDZ for x ends here
    console.log(x); // 5
  }
```

Q6

What is the difference between == and ===?

== (Loose Equality): Compares values after performing type coercion if the types differ.

```
0 == "0"    // true  (string "0" is coerced to number 0)
  null == undefined // true
  false == 0  // true
```

=== (Strict Equality): Compares both value AND type with NO coercion.

```
0 === "0"   // false (different types)
  null === undefined // false
  false === 0 // false
```

Best practice: Always use === to avoid unexpected type-coercion bugs.

Q7

What is type coercion in JavaScript?

Type coercion is the automatic or implicit conversion of values from one data type to another. JavaScript performs coercion in many situations: Implicit coercion (automatic):

```
"5" + 3        // "53"  (number coerced to string for +)
  "5" - 3        // 2     (string coerced to number for -)
  true + 1       // 2     (true coerces to 1)
  false + 1      // 1     (false coerces to 0)
  null + 1       // 1     (null coerces to 0)
  undefined + 1  // NaN   (undefined coerces to NaN)
```

Explicit coercion (manual):

```
Number("42")   // 42
  String(42)     // "42"
  Boolean(0)     // false
  parseInt("3.14") // 3
```

Q8

What are truthy and falsy values?

Falsy values (evaluate to false in boolean context):

```
false, 0, -0, 0n, "" (empty string), null, undefined, NaN
```

Everything else is truthy. Notable truthy values:

```
"0"      // truthy (non-empty string)
  []       // truthy (empty array)
  {}       // truthy (empty object)
  -1       // truthy (non-zero number)
  "false"  // truthy (non-empty string)
```

Example:

```
if ([]) console.log("truthy"); // prints "truthy"
  if (0)  console.log("won't run");
```

Q9

What is NaN and how do you check for it?

NaN stands for "Not a Number". It is a special numeric value representing an undefined or unrepresentable mathematical result. NaN is produced by: - 0 / 0

```
- Math.sqrt(-1)
  - parseInt("hello")
  - undefined + 1 ... (most cases involving undefined in arithmetic)
```

Important quirk: NaN !== NaN (NaN is the only value not equal to itself). Checking for NaN:

```
isNaN("hello")     // true  (but isNaN coerces its argument first – unreliable)
```

```
isNaN(undefined)   // true  (coerces to NaN)
Number.isNaN("hello") // false (no coercion – preferred method)
Number.isNaN(NaN)     // true  (most reliable)
```

Q10

What is the difference between null and undefined?

undefined: - A variable has been declared but not assigned a value

```
- Default return value of a function with no return statement
  - Accessing a missing object property returns undefined
  - typeof undefined === "undefined"
```

null: - An intentional assignment representing "no value" or "empty" - Must be explicitly assigned by the programmer

```
- typeof null === "object" (historical bug in JavaScript)
```

- Used to release an object reference Key difference:

```
null == undefined   // true  (loose equality)
  null === undefined  // false (strict equality)
  null + 1            // 1     (null coerces to 0)
  undefined + 1       // NaN
```

Functions

Q11

What are the different ways to define a function in JavaScript?

1. Function Declaration:

```
function greet(name) { return "Hello " + name; }
  - Hoisted completely (can be called before declaration)
```

2. Function Expression:

```
const greet = function(name) { return "Hello " + name; };
```

- Not hoisted 3. Arrow Function (ES6):

```
const greet = (name) => "Hello " + name;
```

- Shorter syntax, no own 'this', 'arguments', or 'super' 4. Named Function Expression:

```
const greet = function sayHello(name) { return "Hello " + name; };
```

5. Immediately Invoked Function Expression (IIFE):

```
(function() { console.log("Runs immediately"); })();
```

6. Method shorthand (inside an object):

```
const obj = { greet() { return "Hello"; } };
```

7. Constructor Function:

```
function Person(name) { this.name = name; }
  const p = new Person("Alice");
```

8. Generator Function:

```
function* gen() { yield 1; yield 2; }
```

Q12

What is a closure in JavaScript?

A closure is a function that has access to its own scope, the outer function's scope, and the global scope – even after the outer function has finished executing. A closure "closes over" the variables of its surrounding scope. Example:

```
function makeCounter() {
    let count = 0;
    return function() {
```

count++;

```
return count;
    };
```

```
}
const counter = makeCounter();
counter(); // 1
counter(); // 2
counter(); // 3
```

Use cases: - Data privacy / encapsulation

```
- Factory functions
```

- Memoization - Partial application / currying - Event handlers and callbacks that retain state

---

Q13

What is the difference between a regular function and an arrow function?

1. this binding:

```
- Regular function: 'this' is dynamic, determined by how the function is called
   - Arrow function: 'this' is lexically inherited from the surrounding scope
```

2. arguments object:

```
- Regular function: has its own 'arguments' object
   - Arrow function: does NOT have 'arguments' (use rest parameters instead)
```

3. Constructor:

```
- Regular function: can be used as a constructor with 'new'
   - Arrow function: CANNOT be used as a constructor (throws TypeError)
```

4. prototype property:

```
- Regular function: has a prototype property
   - Arrow function: does NOT have a prototype property
```

5. Syntax:

```
- Regular: function(a, b) { return a + b; }
   - Arrow:   (a, b) => a + b
```

Example of 'this' difference:

```
const obj = {
```

name: "Alice",

```
greetRegular: function() { return this.name; }, // "Alice"
    greetArrow: () => this.name,                    // undefined (this is global/window)
  };
```

Q14

What is a higher-order function?

A higher-order function is a function that either:

```
1. Takes one or more functions as arguments, OR
  2. Returns a function as its result (or both)
```

Common built-in higher-order functions:

```
- Array.prototype.map()
  - Array.prototype.filter()
  - Array.prototype.reduce()
  - Array.prototype.forEach()
  - setTimeout()
```

Example:

```
// Takes a function as argument
  const numbers = [1, 2, 3, 4, 5];
  const doubled = numbers.map(n => n * 2); // [2, 4, 6, 8, 10]

// Returns a function
  function multiplier(factor) {
    return (number) => number * factor;
  }
  const triple = multiplier(3);
  triple(5); // 15
```

Q15

What is currying in JavaScript?

Currying is a technique of transforming a function with multiple arguments into a sequence of functions, each taking a single argument. Example:

```
// Normal function
  function add(a, b, c) { return a + b + c; }

// Curried version
  function curriedAdd(a) {
    return function(b) {
      return function(c) {
        return a + b + c;
      };
    };
  }
  curriedAdd(1)(2)(3); // 6

// Using arrow functions
  const curriedAdd = a => b => c => a + b + c;
```

Benefits:

```
- Create specialized/partial functions easily
```

- Improve code reusability - Functional composition

Q16

What is the difference between call, apply, and bind?

All three are used to set the 'this' context of a function explicitly. call(thisArg, arg1, arg2, ...):

```
- Invokes the function immediately
```

- Arguments passed individually

```
function greet(greeting) { return greeting + " " + this.name; }
  greet.call({ name: "Alice" }, "Hello"); // "Hello Alice"
```

apply(thisArg, [argsArray]):

```
- Invokes the function immediately
```

- Arguments passed as an array

```
greet.apply({ name: "Bob" }, ["Hi"]); // "Hi Bob"
```

bind(thisArg, arg1, arg2, ...):

```
- Returns a NEW function with 'this' bound (does NOT invoke immediately)
```

- Can also partially apply arguments

```
const boundGreet = greet.bind({ name: "Carol" });
  boundGreet("Hey"); // "Hey Carol"
```

Memory aid: Call = Comma-separated, Apply = Array, Bind = returns Bound function

---

Q17

What is an IIFE (Immediately Invoked Function Expression)?

An IIFE is a function that is defined and immediately executed. It creates a private scope and was widely used before ES6 modules to avoid polluting the global namespace. Syntax:

```
(function() {
    // code here
    var privateVar = "I'm private";
  })();

// Arrow function IIFE
  (() => {
    console.log("I run immediately");
  })();

// With arguments
  (function(a, b) {
    console.log(a + b);
  })(5, 3); // 8
```

Use cases: - Avoid global namespace pollution - Create private variables - Initialization code that runs once

```
- Module pattern (before ES6 modules)
```

Q18

What are default parameters in JavaScript?

Default parameters allow function parameters to be initialized with default values if no value or undefined is passed. Syntax (ES6+):

```
function greet(name = "World") {
    return "Hello, " + name + "!";
```

```
}
greet();         // "Hello, World!"
greet("Alice");  // "Hello, Alice!"
greet(undefined); // "Hello, World!" (undefined triggers default)
greet(null);      // "Hello, null!"  (null does NOT trigger default)
```

Default expressions (can use previous parameters):

```
function createURL(path, base = "https://example.com", full = base + path) {
    return full;
}
```

Q19

What are rest parameters and the spread operator?

Rest Parameters (...args): - Collects multiple arguments into a single array - Must be the LAST parameter

```
- Replaces the old 'arguments' object (works in arrow functions too)
```

```
function sum(...nums) {
    return nums.reduce((acc, n) => acc + n, 0);
}
sum(1, 2, 3, 4); // 10
```

Spread Operator (...):

```
- Expands an iterable (array, string, etc.) into individual elements
```

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combined = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]
```

```
// Copy an array
const copy = [...arr1]; // [1, 2, 3]
```

```
// Spread in function call
Math.max(...arr1); // 3
```

```
// Spread with objects
const obj1 = { a: 1 };
const obj2 = { ...obj1, b: 2 }; // { a: 1, b: 2 }
```

Q20

What is memoization?

Memoization is an optimization technique that stores the results of expensive function calls and returns the cached result when the same inputs occur again. Example:

```
function memoize(fn) {
    const cache = {};
    return function(...args) {
      const key = JSON.stringify(args);
      if (key in cache) {
        return cache[key];
      }
```

```
      cache[key] = fn.apply(this, args);
      return cache[key];
    };
  }

function slowFactorial(n) {
    return n <= 1 ? 1 : n * slowFactorial(n - 1);
  }

const factorial = memoize(slowFactorial);
  factorial(5); // computed: 120
  factorial(5); // cached: 120 (instant)
```

Benefits:

```
- Improves performance for pure functions with expensive computation
  - Reduces redundant calculations (e.g., Fibonacci, API calls)
```

Scope and this

Q21

What is scope in JavaScript?

Scope determines the accessibility (visibility) of variables in JavaScript. There are four types of scope: 1. Global Scope: Variables declared outside any function/block; accessible everywhere.

```
var globalVar = "I'm global";
```

2. Function Scope: Variables declared inside a function are accessible only within that function (var).

```
function foo() { var x = 10; } // x not accessible outside
```

3. Block Scope (ES6): Variables declared inside a block {} with let/const are only accessible within that block.

```
{ let blockVar = 1; } // blockVar not accessible outside
```

4. Module Scope: Variables in an ES6 module are scoped to that module. Scope Chain: When a variable is accessed, JavaScript looks up the scope chain (current scope → outer scope → ... → global scope) until found or error is thrown.

Q22

What is the 'this' keyword in JavaScript?

'this' refers to the object that is currently executing the code. Its value depends on how a function is called: 1. Global context: this = window (browser) or global (Node.js)

```
console.log(this); // Window
```

2. Method call: this = the object the method is called on

```
const obj = { name: "Alice", getName() { return this.name; } };
   obj.getName(); // "Alice"
```

3. Regular function (non-strict): this = global object

```
function show() { console.log(this); } // Window
```

4. Regular function (strict mode): this = undefined

```
"use strict"; function show() { console.log(this); } // undefined
```

5. Arrow function: this = inherited from enclosing lexical scope 6. Constructor (new): this = the newly created object

```
function Person(n) { this.name = n; }
   new Person("Bob"); // { name: "Bob" }
```

7. Explicit binding (call/apply/bind): this = first argument passed

---

Q23

What is the scope chain?

The scope chain is the hierarchy of scopes that JavaScript uses to resolve variable names. When a variable is used, JavaScript starts looking in the current scope, then moves outward to parent scopes, until it reaches the global scope. Example:

```
const global = "global";
  function outer() {
    const outerVar = "outer";
    function inner() {
      const innerVar = "inner";
      console.log(innerVar); // "inner"    (own scope)
      console.log(outerVar); // "outer"    (outer scope)
      console.log(global);   // "global"   (global scope)
    }
    inner();
  }
  outer();
```

If a variable is not found anywhere in the chain, a ReferenceError is thrown.

Q24

What is lexical scope?

Lexical scope (also called static scope) means that the scope of a variable is determined by where it is written in the source code (at authoring/compile time), NOT by where it is called at runtime. In JavaScript, inner functions have access to variables in their outer function's scope because of lexical scoping:

```
function outer() {
    let x = 10;
    function inner() {
      console.log(x); // 10 – x is lexically in scope
    }
    inner();
  }
```

Arrow functions use lexical 'this' – they inherit 'this' from the surrounding lexical scope, which is why they're often preferred in callbacks.

Objects and Prototypes

Q25

What are the different ways to create an object in JavaScript?

1. Object Literal:

```
const obj = { name: "Alice", age: 30 };
```

2. Object Constructor:

```
const obj = new Object();
```

obj.name = "Alice"; 3. Object.create():

```
const proto = { greet() { return "Hello " + this.name; } };
  const obj = Object.create(proto);
```

obj.name = "Alice"; 4. Constructor Function:

```
function Person(name) { this.name = name; }
  const p = new Person("Alice");
```

5. ES6 Class:

```
class Person { constructor(name) { this.name = name; } }
  const p = new Person("Alice");
```

6. Factory Function:

```
function createPerson(name) { return { name, greet() { return "Hi " + this.name; } }; }
```

Q26

What is prototypal inheritance in JavaScript?

JavaScript uses prototypal inheritance: every object has an internal link to another object called its prototype. When a property is accessed, JS first looks at the object itself, then traverses up the prototype chain until the property is found or null is reached. Example:

```
function Animal(name) { this.name = name; }
  Animal.prototype.speak = function() { return this.name + " makes a noise."; };

function Dog(name) { Animal.call(this, name); }
  Dog.prototype = Object.create(Animal.prototype);
```

Dog.prototype.constructor = Dog;

```
Dog.prototype.bark = function() { return this.name + " barks."; };

const d = new Dog("Rex");
  d.speak(); // "Rex makes a noise."  (inherited from Animal.prototype)
  d.bark();  // "Rex barks."
```

Prototype chain: d → Dog.prototype → Animal.prototype → Object.prototype → null

Q27

What is the difference between __proto__ and prototype?

prototype: - A property on constructor FUNCTIONS - Used to set what will become the __proto__ of instances created by that constructor

```
- function Foo() {} → Foo.prototype is the prototype of objects created with new Foo()
```

__proto__ (or Object.getPrototypeOf()): - A property on every OBJECT INSTANCE

```
- Points to the object's prototype (what it inherits from)
  - Not recommended to use __proto__ directly; use Object.getPrototypeOf() instead
```

Example:

```
function Car(make) { this.make = make; }
  const tesla = new Car("Tesla");
  tesla.__proto__ === Car.prototype;         // true
  Object.getPrototypeOf(tesla) === Car.prototype; // true (preferred)
```

Q28

What is Object.create() and how does it differ from new?

Object.create(proto):

```
- Creates a new object with the specified object as its prototype
  - Does NOT call a constructor function
```

- Returns an object whose __proto__ is set to proto

```
const animal = { type: "Animal", describe() { return this.type; } };
  const dog = Object.create(animal);
```

dog.type = "Dog";

```
dog.describe(); // "Dog"
```

new Keyword:

```
- Calls a constructor function
  - Creates a new object, sets its __proto__ to Constructor.prototype
  - Sets 'this' in the constructor to the new object
  - Returns the new object (unless constructor returns a non-primitive)
```

Key difference: Object.create gives full control over the prototype, while new follows the constructor pattern.

Q29

What is Object destructuring?

Object destructuring is a syntax that allows extracting properties from objects into distinct variables. Basic:

```
const { name, age } = { name: "Alice", age: 30 };
  // name = "Alice", age = 30
```

Rename (alias):

```
const { name: firstName } = { name: "Alice" };
  // firstName = "Alice"
```

Default values:

```
const { name = "Unknown", role = "User" } = { name: "Bob" };
  // name = "Bob", role = "User"
```

Nested:

```
const { address: { city } } = { address: { city: "Mumbai" } };
  // city = "Mumbai"
```

Rest in destructuring:

```
const { a, ...rest } = { a: 1, b: 2, c: 3 };
  // a = 1, rest = { b: 2, c: 3 }
```

In function parameters:

```
function display({ name, age = 0 }) { console.log(name, age); }
  display({ name: "Alice", age: 25 }); // Alice 25
```

Q30

What are getters and setters in JavaScript?

Getters and setters are special methods that get and set the value of an object property. They allow you to execute code when a property is accessed or modified. Object Literal Syntax:

```
const person = {
```

_name: "Alice",

```
get name() { return this._name.toUpperCase(); },
    set name(val) {
      if (typeof val !== "string") throw new Error("Name must be a string");
```

this._name = val;

```
}
  };
  person.name;         // "ALICE"  (getter called)
  person.name = "Bob"; // (setter called)
```

Class Syntax:

```
class Circle {
    constructor(radius) { this._radius = radius; }
    get area() { return Math.PI * this._radius ** 2; }
    set radius(r) { if (r < 0) throw new Error("Negative radius"); this._radius = r; }
  }
```

Q31

What are the methods available on Object?

Commonly used Object static methods: Object.keys(obj) – returns array of own enumerable property NAMES Object.values(obj) – returns array of own enumerable property VALUES Object.entries(obj) – returns array of [key, value] pairs Object.assign(target, ...sources) – copies properties to target (shallow) Object.freeze(obj) – makes object immutable (no add/delete/modify) Object.seal(obj) – prevents add/delete, but allows modify Object.create(proto) – creates object with given prototype Object.defineProperty(obj, prop, descriptor) – define/modify property with full control Object.getPrototypeOf(obj) – returns prototype Object.setPrototypeOf(obj, proto) – sets prototype Object.hasOwn(obj, key) – (ES2022) checks own property (preferred over hasOwnProperty) Object.fromEntries(entries) – creates object from [key, value] array (inverse of entries) Example:

```
const obj = { a: 1, b: 2, c: 3 };
  Object.keys(obj);    // ["a", "b", "c"]
  Object.values(obj);  // [1, 2, 3]
  Object.entries(obj); // [["a",1], ["b",2], ["c",3]]
```

Q32

What is the 'new' keyword and what does it do?

The 'new' keyword creates an instance of a constructor function or class. It does the following steps: 1. Creates a new empty object: {} 2. Sets the __proto__ of the new object to Constructor.prototype 3. Binds 'this' inside the constructor to the new object 4. Executes the constructor function body 5. Returns the new object (unless the constructor explicitly returns a non-primitive) Example:

```
function Car(make, model) {
```

this.make = make; this.model = model;

```
}
  Car.prototype.describe = function() { return this.make + " " + this.model; };

const tesla = new Car("Tesla", "Model 3");
  tesla.describe(); // "Tesla Model 3"
  tesla instanceof Car; // true
```

If the constructor returns an object, that object is returned instead of 'this'.

Arrays

Q33

What are the most important Array methods in JavaScript?

Mutating methods (change original array):

```
push(el)         – add to end, returns new length
  pop()            – remove from end, returns removed element
  shift()          – remove from start, returns removed element
  unshift(el)      – add to start, returns new length
  splice(i,n,...els) – remove/replace/insert elements at index
  sort(fn)         – sort in place
  reverse()        – reverse in place
  fill(val,s,e)    – fill with value from s to e
```

Non-mutating methods (return new value/array):

```
map(fn)          – transform each element, returns new array
  filter(fn)       – keep elements where fn returns truthy
  reduce(fn, init) – reduce to single value
  find(fn)         – first element where fn is truthy
  findIndex(fn)    – index of first match
  every(fn)        – true if ALL elements pass fn
  some(fn)         – true if ANY element passes fn
  includes(val)    – true if val is in array
  indexOf(val)     – index of val (-1 if not found)
  slice(s, e)      – copy from s to e (exclusive)
  flat(depth)      – flatten nested arrays
  flatMap(fn)      – map then flatten one level
  concat(...arrs)  – merge arrays
  join(sep)        – join to string
  forEach(fn)      – iterate (returns undefined)
```

Q34

What is the difference between map, filter, and reduce?

map(fn): - Transforms each element using fn - Returns a NEW array of the SAME length

```
const nums = [1, 2, 3];
  nums.map(n => n * 2); // [2, 4, 6]
```

filter(fn):

```
- Keeps elements for which fn returns truthy
```

- Returns a NEW array of SAME or SMALLER length

```
nums.filter(n => n > 1); // [2, 3]
```

reduce(fn, initialValue): - Accumulates array into a SINGLE value

```
- fn receives (accumulator, currentValue, index, array)
  nums.reduce((acc, n) => acc + n, 0); // 6
```

Chaining:

```
[1, 2, 3, 4, 5]
    .filter(n => n % 2 === 0)  // [2, 4]
    .map(n => n * n)            // [4, 16]
    .reduce((a, b) => a + b, 0); // 20
```

Q35

What is the difference between for...of and for...in?

for...in:

```
- Iterates over ENUMERABLE PROPERTY KEYS (strings) of an object
  - Also iterates inherited enumerable properties (use hasOwnProperty check)
  - Should NOT be used to iterate arrays (iterates indices as strings, may include inherited props)

const obj = { a: 1, b: 2, c: 3 };
  for (const key in obj) console.log(key); // "a", "b", "c"
```

for...of:

```
- Iterates over ITERABLE VALUES (arrays, strings, Maps, Sets, generators)
  - Does NOT work on plain objects (not iterable by default)

const arr = [10, 20, 30];
  for (const val of arr) console.log(val); // 10, 20, 30

const str = "hello";
  for (const char of str) console.log(char); // h, e, l, l, o
```

Q36

How do you flatten a nested array?

1. Array.prototype.flat(depth):

```
[1, [2, [3, [4]]]].flat();    // [1, 2, [3, [4]]] (depth 1 by default)
  [1, [2, [3, [4]]]].flat(2);   // [1, 2, 3, [4]]
  [1, [2, [3, [4]]]].flat(Infinity); // [1, 2, 3, 4] (fully flat)
```

2. flatMap():

```
[1, 2, 3].flatMap(n => [n, n * 2]); // [1, 2, 2, 4, 3, 6]
```

3. Using reduce + recursion (manual):

```
function deepFlat(arr) {
    return arr.reduce((acc, val) =>
      Array.isArray(val) ? acc.concat(deepFlat(val)) : acc.concat(val), []);
  }
```

4. Using spread + concat (only one level):

```
[].concat(...[1, [2, [3]]]); // [1, 2, [3]]
```

Q37

What is array destructuring?

Array destructuring allows unpacking values from arrays into variables based on position. Basic:

```
const [a, b, c] = [1, 2, 3];
  // a = 1, b = 2, c = 3
```

Skip elements:

```
const [first, , third] = [1, 2, 3];
  // first = 1, third = 3
```

Default values:

```
const [x = 10, y = 20] = [5];
  // x = 5, y = 20
```

Rest:

```
const [head, ...tail] = [1, 2, 3, 4];
  // head = 1, tail = [2, 3, 4]
```

Swap variables:

```
let p = 1, q = 2;
  [p, q] = [q, p];
  // p = 2, q = 1
```

From function return:

```
function minMax(arr) { return [Math.min(...arr), Math.max(...arr)]; }
  const [min, max] = minMax([3, 1, 4, 1, 5]);
```

Asynchronous JavaScript

Q38

What is the Event Loop in JavaScript?

JavaScript is single-threaded but can handle asynchronous operations through the event loop. The event loop coordinates: Components:

```
- Call Stack: Executes synchronous code (LIFO)
  - Web APIs: Handle async tasks (setTimeout, fetch, DOM events) off the main thread
  - Callback Queue (Macrotask Queue): Holds callbacks from Web APIs when ready
  - Microtask Queue: Holds Promise callbacks (.then/.catch) – has HIGHER priority than macrotask queue
```

- Event Loop: Monitors call stack; when empty, moves tasks from queues to stack Execution order: 1. All synchronous code in call stack

```
2. All microtasks (Promise callbacks)
  3. One macrotask (setTimeout callback)
```

4. Repeat from step 2 Example:

```
console.log("1");
  setTimeout(() => console.log("4"), 0);
  Promise.resolve().then(() => console.log("3"));
  console.log("2");
  // Output: 1, 2, 3, 4
```

Q39

What is a Promise in JavaScript?

A Promise is an object representing the eventual completion or failure of an asynchronous operation. It has three states: - pending: initial state, neither fulfilled nor rejected - fulfilled: operation completed successfully - rejected: operation failed Creating a Promise:

```
const promise = new Promise((resolve, reject) => {
    // async work
    if (success) resolve(value);
    else reject(error);
  });
```

Consuming: promise

```
.then(value => console.log("Success:", value))
    .catch(error => console.log("Error:", error))
    .finally(() => console.log("Always runs"));
```

Promise static methods:

```
Promise.all([p1, p2, p3])       – resolves when ALL resolve; rejects if ANY rejects
  Promise.allSettled([p1, p2])    – waits for ALL to settle (fulfilled or rejected)
```

```
Promise.race([p1, p2])          – settles with the FIRST to settle
Promise.any([p1, p2])           – resolves with FIRST fulfilled; rejects if ALL reject
Promise.resolve(val)            – returns already resolved promise
Promise.reject(err)             – returns already rejected promise
```

Q40

What is async/await?

async/await is syntactic sugar built on top of Promises, making asynchronous code look and behave like synchronous code. async function:

```
- Always returns a Promise
  - Allows use of await keyword inside
```

await:

```
- Pauses execution of the async function until the Promise resolves
```

- Returns the resolved value

```
- Can only be used inside an async function (or top-level module)
```

Example:

```
async function fetchUser(id) {
    try {
      const response = await fetch(`/api/users/${id}`);
      if (!response.ok) throw new Error("User not found");
      const user = await response.json();
      return user;
    } catch (error) {
      console.error(error);
    }
  }
```

Parallel execution with async/await:

```
// Sequential (slower):
  const a = await fetchA();
  const b = await fetchB();

// Parallel (faster):
  const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

Q41

What is the difference between microtasks and macrotasks?

Macrotasks (Task Queue):

```
- setTimeout, setInterval, setImmediate (Node.js)
```

- I/O operations, UI rendering - One macrotask is processed per event loop iteration Microtasks (Microtask Queue):

```
- Promise.then/.catch/.finally callbacks
  - queueMicrotask()
```

- MutationObserver callbacks - ALL microtasks are processed before the next macrotask Order example:

```
console.log("sync");           // 1
  setTimeout(() => {
    console.log("macrotask");    // 4
  }, 0);
  Promise.resolve()
    .then(() => console.log("microtask 1")) // 3
    .then(() => console.log("microtask 2")); // still microtask – runs before macrotask!
  console.log("sync end");       // 2
  // Output: sync, sync end, microtask 1, microtask 2, macrotask
```

Q42

What is callback hell and how do you avoid it?

Callback hell (also called the "pyramid of doom") occurs when multiple nested asynchronous callbacks make code hard to read and maintain. Example of callback hell:

```
getData(function(a) {
    getMoreData(a, function(b) {
      getEvenMore(b, function(c) {
        console.log(c);
      });
    });
  });
```

Solutions: 1. Modularize callbacks into named functions:

```
function handleC(c) { console.log(c); }
   function handleB(b) { getEvenMore(b, handleC); }
   function handleA(a) { getMoreData(a, handleB); }
   getData(handleA);
```

2. Use Promises:

```
getData()
     .then(a => getMoreData(a))
     .then(b => getEvenMore(b))
     .then(c => console.log(c));
```

3. Use async/await:

```
async function run() {
     const a = await getData();
     const b = await getMoreData(a);
     const c = await getEvenMore(b);
     console.log(c);
   }
```

Q43

What are generators in JavaScript?

Generators are special functions that can pause and resume their execution. They are defined using function* syntax and use the yield keyword to pause.

```
function* counter() {
    let i = 0;
    while (true) {
```

yield i++;

```
}
  }
```

```
const gen = counter();
  gen.next(); // { value: 0, done: false }
  gen.next(); // { value: 1, done: false }
  gen.next(); // { value: 2, done: false }
```

Key concepts:

```
- Generator functions return an iterator object
  - yield pauses execution and returns a value
  - .next() resumes execution until the next yield
  - .next(val) can send a value back INTO the generator
  - { value, done } is returned by each .next() call
```

Use cases: - Lazy / infinite sequences - Implementing custom iterators

```
- Async control flow (async generators)
```

- Co-routines

Q44

What is the difference between setTimeout and setInterval?

setTimeout(fn, delay, ...args): - Executes fn ONCE after at least delay milliseconds

```
- Returns a timer ID (use clearTimeout(id) to cancel)
  setTimeout(() => console.log("once"), 1000); // after 1s
```

setInterval(fn, delay, ...args): - Executes fn REPEATEDLY every delay milliseconds

```
- Returns a timer ID (use clearInterval(id) to cancel)
  const id = setInterval(() => console.log("repeat"), 1000);
  clearInterval(id); // stop it
```

Note: The delay is a MINIMUM, not a guarantee. If the call stack is busy, execution is delayed. Implementing setInterval with setTimeout (more accurate):

```
function accurateInterval(fn, delay) {
    function loop() {
      fn();
      setTimeout(loop, delay);
    }
    setTimeout(loop, delay);
  }
```

ES6+ Features

Q45

What are template literals?

Template literals (introduced in ES6) are string literals that allow embedded expressions, multiline strings, and tagged templates. They use backticks (`) instead of quotes. Basic usage:

```
const name = "Alice";
  const greeting = `Hello, ${name}!`; // "Hello, Alice!"
```

Expression interpolation:

```
const a = 5, b = 10;
  console.log(`Sum is ${a + b}`); // "Sum is 15"
```

Multiline strings:

```
const multiline = `Line 1
```

Line 2

```
Line 3`; // no need for \n
```

Tagged templates:

```
function highlight(strings, ...values) {
    return strings.reduce((result, str, i) =>
      result + str + (values[i] ? `<strong>${values[i]}</strong>` : ""), "");
  }
  highlight`Name: ${name} Age: ${30}`; // "Name: <strong>Alice</strong> Age: <strong>30</strong>
```

Q46

What are symbols in JavaScript?

Symbol is a primitive data type introduced in ES6. Every Symbol value is unique and immutable. Symbols are often used as unique object property keys. Creating symbols:

```
const sym1 = Symbol();
  const sym2 = Symbol("description");
  sym1 === sym2; // false (always unique)
  sym2.toString(); // "Symbol(description)"
```

As object keys:

```
const ID = Symbol("id");
  const user = { [ID]: 123, name: "Alice" };
  user[ID]; // 123
  Object.keys(user); // ["name"] — symbol keys are not enumerable
```

Well-known symbols:

```
Symbol.iterator  – makes objects iterable (for...of)
```

Symbol.toPrimitive – customizes type conversion Symbol.hasInstance – customizes instanceof Symbol.toStringTag – customizes Object.prototype.toString output Global symbol registry:

```
const s = Symbol.for("shared");
  Symbol.for("shared") === s; // true — same symbol reused
```

Q47

What are Proxy and Reflect in JavaScript?

Proxy:

```
A Proxy wraps an object and intercepts fundamental operations like property access, assignment, enumeratio
```

```
const handler = {
    get(target, prop) {
      return prop in target ? target[prop] : `Property ${prop} not found`;
    },
    set(target, prop, value) {
      if (typeof value !== "number") throw new TypeError("Only numbers allowed");
      target[prop] = value;
      return true;
    }
  };
```

```
const proxy = new Proxy({}, handler);
  proxy.age = 25;    // OK
  proxy.name;        // "Property name not found"
```

Available traps: get, set, has, deleteProperty, apply, construct, ownKeys, etc. Reflect: Provides methods mirroring the Proxy traps. Used to perform the default behaviour inside traps.

```
Reflect.get(target, prop);
  Reflect.set(target, prop, value);
  Reflect.has(target, prop);
```

Q48

What are Map and Set in JavaScript?

Map:

```
- Key-value store where keys can be ANY type (objects, functions, primitives)
```

- Maintains insertion order - Better performance than objects for frequent add/remove

```
const map = new Map();
  map.set("name", "Alice");
  map.set(42, "the answer");
  map.get("name");    // "Alice"
  map.has(42);        // true
  map.size;           // 2
  map.delete("name");
  map.clear();
```

Set: - Stores UNIQUE values of any type - Maintains insertion order

```
const set = new Set([1, 2, 2, 3, 3]);
  set;        // Set {1, 2, 3}
  set.add(4);
  set.has(2); // true
  set.size;   // 4
  set.delete(1);

// Remove duplicates from array:
  const unique = [...new Set([1, 2, 2, 3])]; // [1, 2, 3]
```

WeakMap / WeakSet: Like Map/Set but keys must be objects and are weakly referenced (eligible for garbage collection if no other references).

Q49

What are iterators and iterables in JavaScript?

Iterator Protocol:

```
An object is an iterator if it has a next() method that returns { value, done }.

const iterator = {
```

current: 1, last: 5,

```
next() {
      return this.current <= this.last
        ? { value: this.current++, done: false }
        : { value: undefined, done: true };
    }
  };
  iterator.next(); // { value: 1, done: false }
```

Iterable Protocol:

```
An object is iterable if it has a [Symbol.iterator] method that returns an iterator.
```

Built-in iterables: Arrays, Strings, Maps, Sets, generator objects.

```
const range = {
```

from: 1, to: 5,

```
[Symbol.iterator]() {
      let current = this.from;
      const last = this.to;
      return { next() {
        return current <= last
          ? { value: current++, done: false }
          : { done: true };
      }};
    }
  };
  for (const num of range) console.log(num); // 1 2 3 4 5
```

Q50

What are WeakMap and WeakSet?

WeakMap:

```
- Like Map, but keys MUST be objects (not primitives)
```

- Keys are held "weakly" – if no other reference to the key object exists, it can be garbage collected

```
- Not enumerable (no .keys(), .values(), .entries(), or .size)
```

```
const wm = new WeakMap();
  let obj = {};
  wm.set(obj, "data");
  wm.get(obj); // "data"
  obj = null;  // obj can now be garbage collected, WeakMap entry disappears
```

WeakSet: - Like Set, but values MUST be objects - Values are weakly referenced - Not enumerable

```
const ws = new WeakSet();
  let o = {};
  ws.add(o);
  ws.has(o); // true
  o = null;  // garbage collected
```

Use cases: - Storing metadata/private data associated with objects without preventing GC - Tracking which objects have been processed - Caching computed results per object

---

Classes and OOP

Q51

What are ES6 classes in JavaScript?

ES6 classes are syntactic sugar over JavaScript's existing prototype-based inheritance. They provide a cleaner, more familiar syntax for creating objects and dealing with inheritance.

```
class Animal {
    constructor(name) {
```

this.name = name;

```
}
    speak() {
      return `${this.name} makes a noise.`;
    }
    static create(name) { // static method
      return new Animal(name);
    }
  }
class Dog extends Animal {
    constructor(name) {
      super(name); // must call super() before using 'this'
    }
    speak() {
      return `${this.name} barks.`;
    }
  }
const d = new Dog("Rex");
  d.speak();           // "Rex barks."
  d instanceof Dog;    // true
  d instanceof Animal; // true
```

Key features:

```
- constructor() method
  - Method definitions (no 'function' keyword)
  - static methods (called on the class, not instances)
```

- extends for inheritance

```
- super() to call parent constructor/methods
  - Private fields with # prefix (ES2022)
```

Q52

What are private class fields?

Private class fields (introduced in ES2022) use the # prefix and are only accessible within the class body. They cannot be accessed from outside the class or in subclasses.

```
class BankAccount {
    #balance = 0;  // private field

constructor(initialBalance) {
```

this.#balance = initialBalance;

```
}

deposit(amount) {
      if (amount > 0) this.#balance += amount;
    }

get balance() {
      return this.#balance;
    }
  }

const account = new BankAccount(1000);
  account.balance;   // 1000 (via getter)
  account.#balance;  // SyntaxError: Private field '#balance' must be declared in an enclosing class
```

Private methods:

```
class Example {
    #privateMethod() { return "private"; }
    publicMethod() { return this.#privateMethod(); }
  }
```

Q53

What is the difference between static and instance methods?

Instance methods:

```
- Defined on the class without 'static' keyword
  - Called on instances (objects created with new)
  - Have access to 'this' (the instance)

class Counter {
    constructor() { this.count = 0; }
    increment() { this.count++; } // instance method
  }
  const c = new Counter();
  c.increment(); // OK
  Counter.increment(); // TypeError: not a function on the class
```

Static methods: - Defined with the 'static' keyword - Called on the CLASS itself, not instances - 'this' refers to the class, not an instance

```
class MathUtils {
    static add(a, b) { return a + b; }
  }
  MathUtils.add(2, 3); // 5
```

```
new MathUtils().add(2, 3); // TypeError
```

Common use cases for static: utility functions, factory methods, constants.

Q54

What are the four pillars of Object-Oriented Programming in JS?

1. Encapsulation: Bundling data and methods that operate on that data within one unit, and restricting direct access to some components.

```
- Private fields (#field), getters/setters, closures
```

2. Abstraction: Hiding complex implementation details and exposing only necessary interfaces. - Classes expose public methods; internals are hidden 3. Inheritance: A class (child) can inherit properties and methods from another class (parent).

```
class Dog extends Animal { ... }
```

4. Polymorphism: Different classes can be treated as instances of the same class through a shared interface. Methods can be overridden in subclasses.

```
class Animal { speak() { return "..."; } }
  class Dog extends Animal { speak() { return "Woof"; } }
  class Cat extends Animal { speak() { return "Meow"; } }

const animals = [new Dog(), new Cat()];
  animals.forEach(a => console.log(a.speak())); // "Woof", "Meow"
  // Same .speak() call, different behaviour – polymorphism!
```

Error Handling

Q55

How does error handling work in JavaScript?

JavaScript uses try...catch...finally blocks for synchronous error handling.

```
try {
    // Code that might throw
    const result = JSON.parse("invalid json");
  } catch (error) {
    // Handle the error
    console.log(error.name);    // "SyntaxError"
    console.log(error.message); // "Unexpected token i..."
  } finally {
    // Always executes (cleanup code)
    console.log("Done");
  }
```

Error types: - Error – base error type - SyntaxError – invalid syntax - TypeError – wrong type - ReferenceError – undefined variable reference

```
- RangeError   – value out of range (e.g., array length)
```

- URIError – malformed URI

```
- EvalError    – relates to eval()
```

Custom errors:

```
class ValidationError extends Error {
    constructor(message, field) {
      super(message);
```

this.name = "ValidationError"; this.field = field;

```
}
  }
  throw new ValidationError("Required", "email");
```

Q56

How do you handle errors in async/await?

Method 1: try/catch (most common):

```
async function fetchData() {
    try {
      const res = await fetch("/api/data");
      if (!res.ok) throw new Error(`HTTP error! status: ${res.status}`);
      return await res.json();
    } catch (err) {
```

```
    console.error("Fetch failed:", err.message);
    return null;
  }
}
```

Method 2: .catch() on the async function call:

```
const data = await fetchData().catch(err => {
  console.error(err);
  return null;
});
```

Method 3: Helper function that wraps promise to avoid repetitive try/catch:

```
async function to(promise) {
  try {
    const data = await promise;
    return [null, data];
  } catch (err) {
    return [err, null];
  }
}
const [err, user] = await to(getUser(id));
if (err) { /* handle */ }
```

Modules

Q57

What are ES6 modules?

ES6 modules provide a native way to organize and share code across files using import and export statements.
Named exports (a file can have many):

```
// math.js
  export const PI = 3.14159;
  export function add(a, b) { return a + b; }
  export function subtract(a, b) { return a - b; }
```

Default export (a file can have only one):

```
// logger.js
  export default function log(msg) { console.log(msg); }
```

Importing named exports:

```
import { PI, add } from "./math.js";
  import { add as sum } from "./math.js"; // alias
```

Importing default export:

```
import log from "./logger.js"; // name can be anything
```

Import everything:

```
import * as Math from "./math.js";
  Math.add(1, 2);
```

Re-exporting:

```
export { add, subtract } from "./math.js";
  export { default as log } from "./logger.js";
```

Key features:

```
- Static analysis (tree-shaking possible)
```

- Strict mode by default

```
- Top-level await supported (in modern environments)
```

Q58

What is the difference between CommonJS and ES Modules?

CommonJS (Node.js traditional):

```
const fs = require("fs");        // import
  module.exports = { myFunc };     // export
  exports.helper = function() {};  // export individual
```

```
  - Dynamic (require can be called conditionally)
```

- Synchronous loading

```
  - .js files in Node.js by default (without "type":"module")
    - exports object is mutable
```

ES Modules (ESM):

```
  import fs from "fs";             // import
    export function myFunc() {}     // export

  - Static (imports resolved at parse time)
```

- Asynchronous loading - .mjs files or "type":"module" in package.json

```
  - Live bindings (imported values reflect changes in the exporting module)
```

Key difference: CommonJS copies the exported value, while ESM creates live bindings. ESM enables tree-shaking by static analysis.

DOM and Browser APIs

Q59

What is the DOM?

The Document Object Model (DOM) is a programming interface for web documents. It represents the HTML document as a tree of nodes/objects that JavaScript can read and manipulate. Selecting elements:

```
document.getElementById("id")
  document.querySelector(".class")         // first match
  document.querySelectorAll("div.item")    // NodeList of all matches
  document.getElementsByClassName("cls")
  document.getElementsByTagName("p")
```

Creating/modifying:

```
const el = document.createElement("div");
```

el.textContent = "Hello"; el.className = "container";

```
el.setAttribute("data-id", "123");
  document.body.appendChild(el);
```

Removing:

```
el.remove();
  parent.removeChild(el);
```

Traversal: el.parentElement

```
el.children          // HTMLCollection
```

el.firstElementChild el.nextElementSibling

Q60

What is event bubbling and capturing?

When an event fires on an element, it goes through three phases: 1. Capture phase: Event travels from root (window) DOWN to the target element 2. Target phase: Event is at the target element 3. Bubble phase: Event travels from target element UP to the root addEventListener(type, handler, useCapture):

```
- useCapture = false (default): handler fires in BUBBLE phase
```

- useCapture = true: handler fires in CAPTURE phase Example:

```
// clicks on inner div will also trigger outer div's handler
```

Click me

```
document.getElementById("outer").addEventListener("click", () => {
    console.log("outer"); // fires second (bubbling)
  });
  document.getElementById("inner").addEventListener("click", () => {
```

```
    console.log("inner"); // fires first
  });
```

Stopping propagation:

```
event.stopPropagation()      – stops bubbling/capturing
  event.stopImmediatePropagation() – also prevents other handlers on same element
  event.preventDefault()       – prevents default browser behavior (not propagation)
```

Q61

What is event delegation?

Event delegation is a pattern where a single event listener is placed on a parent element to handle events from its children (using bubbling). This is efficient for dynamic content and large lists. Instead of:

```
// Bad – adding listener to each item
  document.querySelectorAll(".btn").forEach(btn => {
    btn.addEventListener("click", handler);
  });
```

Use delegation:

```
// Good – one listener on parent
  document.getElementById("list").addEventListener("click", function(e) {
    if (e.target.matches(".btn")) {
      console.log("Button clicked:", e.target.textContent);
    }
    if (e.target.closest(".card")) {
      // handle click anywhere in a card
    }
  });
```

Benefits: - Works for dynamically added elements

```
- Reduces memory usage (fewer event listeners)
```

- Simpler code

Q62

What is the difference between localStorage, sessionStorage, and cookies?

localStorage:

```
- Persists data with NO expiry (until manually cleared)
```

- Accessible by all windows/tabs of same origin - Capacity: ~5-10 MB - Not sent with HTTP requests sessionStorage:

```
- Data persists for the duration of the page SESSION (cleared when tab closes)
  - Per-tab scope (not shared between tabs)
```

- Capacity: ~5-10 MB - Not sent with HTTP requests Cookies:

```
- Can have expiry date (or session-based)
```

- Capacity: ~4 KB - Automatically sent with EVERY HTTP request to the server

```
- Accessible from server and client (unless HttpOnly flag is set)
```

- Support Secure, HttpOnly, SameSite flags for security API comparison:

```
localStorage.setItem("key", "val");
  localStorage.getItem("key");
  localStorage.removeItem("key");
  localStorage.clear();
```

Q63

What is the Fetch API?

The Fetch API provides an interface for making HTTP requests. It returns Promises and is more powerful and flexible than XMLHttpRequest. Basic GET request:

```
fetch("https://api.example.com/data")
    .then(res => res.json())
    .then(data => console.log(data))
    .catch(err => console.error(err));
```

With async/await:

```
async function getData() {
    const res = await fetch("https://api.example.com/data");
    if (!res.ok) throw new Error(`HTTP error! status: ${res.status}`);
    return await res.json();
  }
```

POST request:

```
const response = await fetch("/api/users", {
```

method: "POST",

```
headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: "Alice", age: 30 })
  });
```

Response methods:

```
res.json()    – parse body as JSON (returns Promise)
  res.text()    – parse body as text
  res.blob()    – parse as Blob (binary)
```

res.ok – true if status 200-299 res.status – HTTP status code res.headers – Headers object

---

Design Patterns and Best Practices

Q64

What is debouncing and throttling?

Both are techniques to control how often a function is called, commonly used with events like scroll, resize, or keyup. Debouncing:

```
- Delays executing a function until after a specified time has passed SINCE THE LAST CALL
  - Use case: Search input (only fetch after user stops typing)
```

```
function debounce(fn, delay) {
    let timer;
    return function(...args) {
      clearTimeout(timer);
      timer = setTimeout(() => fn.apply(this, args), delay);
    };
  }
  const debouncedSearch = debounce(search, 300);
  input.addEventListener("input", debouncedSearch);
```

Throttling:

```
- Ensures a function is called AT MOST once in a given time period
```

- Use case: Scroll handler, resize handler

```
function throttle(fn, limit) {
    let lastRun = 0;
    return function(...args) {
      const now = Date.now();
      if (now - lastRun >= limit) {
```

lastRun = now;

```
fn.apply(this, args);
      }
    };
  }
  const throttledScroll = throttle(handleScroll, 100);
```

Q65

What is the Module design pattern?

The Module pattern creates a private scope using closures and exposes only a public API, achieving encapsulation in JavaScript. IIFE-based Module pattern:

```
const Counter = (function() {
    let count = 0; // private
```

```
return {
```

```
      increment() { count++; },
      decrement() { count--; },
      getCount() { return count; }
    };
  })();

Counter.increment();
  Counter.getCount(); // 1
  Counter.count;      // undefined (private!)
```

Revealing Module pattern (cleaner):

```
const Counter = (function() {
    let count = 0;
    function increment() { count++; }
    function getCount() { return count; }
    return { increment, getCount }; // reveal public parts
  })();
```

Modern alternative: ES6 modules achieve this natively with import/export.

Q66

What is the Observer (Pub/Sub) pattern?

The Observer pattern defines a one-to-many relationship where when one object (subject/publisher) changes state, all its dependents (observers/subscribers) are notified automatically. Simple implementation:

```
class EventEmitter {
    constructor() { this.events = {}; }

on(event, listener) {
      if (!this.events[event]) this.events[event] = [];
      this.events[event].push(listener);
    }

off(event, listener) {
      this.events[event] = (this.events[event] || [])
        .filter(l => l !== listener);
    }

emit(event, ...args) {
      (this.events[event] || []).forEach(listener => listener(...args));
    }
  }

const emitter = new EventEmitter();
  emitter.on("data", (val) => console.log("Received:", val));
  emitter.emit("data", 42); // "Received: 42"
```

Used in: Node.js EventEmitter, React event systems, Redux, browser DOM events.

Q67

What is the Singleton pattern?

The Singleton pattern ensures a class has only ONE instance and provides a global access point to it.

```
class Database {
    constructor(connection) {
      if (Database.instance) {
        return Database.instance;
      }
```

this.connection = connection; Database.instance = this;

```
}

query(sql) { /* ... */ }
  }

const db1 = new Database("mongodb://localhost");
  const db2 = new Database("other-connection");
  db1 === db2; // true (same instance!)
```

Using a closure (simpler):

```
const Singleton = (function() {
    let instance;
    function createInstance() { return { id: Math.random() }; }
    return {
      getInstance() {
        if (!instance) instance = createInstance();
        return instance;
      }
    };
  })();
```

Use cases: Database connections, configuration managers, logging services.

Q68

What are pure functions and side effects?

Pure function:

```
- Given the same inputs, always returns the same output
  - Has NO side effects (doesn't modify anything outside its scope)
```

- Does not depend on external state

```
// Pure
  function add(a, b) { return a + b; }
  function square(x) { return x * x; }
```

Side effects are changes to state outside the function's scope:

```
- Modifying a variable outside the function
```

- Mutating arguments - Calling the DOM - Making HTTP requests - Writing to disk/console

```
// Impure (side effect – mutates argument)
  function addToArray(arr, item) { arr.push(item); }

// Pure alternative
```

```
    function addToArray(arr, item) { return [...arr, item]; }
```

Benefits of pure functions:

```
  - Easy to test (no mocking needed)
```

- Predictable and deterministic - Enable memoization - Support parallel/concurrent execution safely

---

Memory Management and Performance

Q69

How does garbage collection work in JavaScript?

JavaScript uses automatic garbage collection to manage memory. The primary algorithm is Mark-and-Sweep: 1. Mark phase: The garbage collector starts from "roots" (global variables, call stack) and marks all reachable objects. 2. Sweep phase: Any objects not marked as reachable are removed from memory. An object becomes eligible for garbage collection when there are NO more references to it:

```
let obj = { name: "Alice" };
  obj = null; // The object { name: "Alice" } is now eligible for GC
```

Memory leaks in JavaScript (common causes):

```
1. Global variables (accidentally created with undeclared variable)
```

2. Event listeners not removed when elements are removed 3. Closures holding references to large objects

```
4. Timers (setInterval) with references to large objects that are never cleared
```

5. DOM references held in JS after the element is removed from DOM How to avoid leaks:

```
- Use const/let, avoid var and global variables
```

- Remove event listeners with removeEventListener - Clear timers with clearTimeout/clearInterval - Use WeakMap/WeakSet for object-keyed caches

---

Q70

What is the difference between shallow copy and deep copy?

Shallow Copy: Copies the top-level properties only. Nested objects/arrays still share the same reference.

```
const original = { name: "Alice", address: { city: "Mumbai" } };
  const shallow = { ...original }; // or Object.assign({}, original)

shallow.name = "Bob";            // OK – doesn't affect original
  shallow.address.city = "Delhi";  // MUTATES original.address.city too!
```

Deep Copy: Creates a completely independent clone, including all nested objects.

```
Method 1 – JSON (simple but limited):
  const deep = JSON.parse(JSON.stringify(original));
  // Limitations: loses undefined, functions, Dates become strings, no circular refs

Method 2 – structuredClone (ES2022, modern):
  const deep = structuredClone(original);
  // Handles: Dates, ArrayBuffers, Sets, Maps; NOT: functions, DOM nodes

Method 3 – recursive function:
  function deepClone(obj) {
    if (obj === null || typeof obj !== "object") return obj;
```

```
  if (Array.isArray(obj)) return obj.map(deepClone);
  return Object.fromEntries(Object.entries(obj).map(([k, v]) => [k, deepClone(v)]));
}
```

Q71

What is virtual DOM and how does it relate to JavaScript performance?

The virtual DOM is a lightweight in-memory representation of the real DOM. Libraries like React use it to optimize rendering: Process:

1. State changes → new virtual DOM tree is created
2. Diffing algorithm (reconciliation) compares new virtual DOM with previous virtual DOM
3. Only the actual differences are applied to the real DOM (minimal DOM operations)

Why DOM operations are expensive:

- DOM manipulation triggers reflow (layout recalculation) and repaint

- Accessing certain properties like offsetHeight forces a synchronous layout Performance tips:

- Batch DOM updates (use document fragments, or library batching)
- Avoid layout thrashing (don't read and write DOM properties in a loop)

- Use requestAnimationFrame for visual updates - Debounce/throttle event-heavy handlers - Use efficient CSS selectors

- Lazy load images and code (code splitting)

Advanced Concepts

Q72

What is the difference between synchronous and asynchronous code?

Synchronous code: - Executes line by line, one at a time - Each operation waits for the previous to complete

```
- Blocks the thread (long operations freeze the UI)
```

```
const result = getDataSync(); // blocks until done
  console.log(result);
```

Asynchronous code: - Does not block the thread

```
- Operations can run in the background (Web APIs)
  - Results are handled via callbacks, promises, or async/await
```

```
getData().then(result => console.log(result)); // non-blocking
  console.log("This runs before result is ready");
```

JavaScript is single-threaded but uses:

```
- Web APIs (browser) / C++ APIs (Node.js) for async operations
```

- Event loop to process results when they're ready

Q73

What is functional programming in JavaScript?

Functional programming (FP) is a programming paradigm that treats computation as the evaluation of mathematical functions, avoiding shared state and mutable data. Core principles:

```
1. Pure functions (same input → same output, no side effects)
  2. Immutability (data is not mutated, new copies are created)
  3. First-class and higher-order functions
```

4. Function composition Function composition:

```
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);
  const pipe    = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);
```

```
const double = x => x * 2;
  const addOne = x => x + 1;
  const square = x => x * x;
```

```
const transform = pipe(double, addOne, square);
  transform(3); // square(addOne(double(3))) = square(7) = 49
```

Partial application:

```
const multiply = (a, b) => a * b;
  const double = multiply.bind(null, 2);
  double(5); // 10
```

Q74

What is a WeakRef in JavaScript?

WeakRef (ES2021) allows holding a weak reference to an object without preventing that object from being garbage collected.

```
let obj = { name: "Alice" };
  const weakRef = new WeakRef(obj);

weakRef.deref(); // { name: "Alice" }

obj = null; // obj can now be garbage collected

// At some later point:
  weakRef.deref(); // undefined (if GC has run)
```

FinalizationRegistry: - Allows registering a callback that runs when an object is garbage collected

```
const registry = new FinalizationRegistry((value) => {
    console.log(value + " was garbage collected");
  });

let target = {};
  registry.register(target, "myObject");
  target = null; // eventually logs "myObject was garbage collected"
```

Use cases: Caches that should not prevent memory release.

Q75

What is strict mode in JavaScript?

Strict mode is a way to opt into a restricted variant of JavaScript that: - Eliminates some JavaScript silent errors by throwing them - Fixes mistakes that make it difficult for JavaScript engines to perform optimizations - Prohibits syntax likely to be defined in future ECMAScript Enable:

```
"use strict"; // at the top of a file or function
  // ES6 modules are always in strict mode
```

What strict mode prevents:

```
- Using undeclared variables: x = 5; // ReferenceError
  - Deleting undeletable properties: delete Object.prototype; // TypeError
  - Duplicate parameter names: function f(a, a) {} // SyntaxError
  - Writing to read-only properties: NaN = 1; // TypeError
  - 'this' in standalone functions is undefined (not window)
```

- with statement is not allowed

```
- eval cannot introduce new variables into surrounding scope
```

Q76

What are tagged template literals?

Tagged templates allow you to process a template literal with a function. The tag function receives the string parts and interpolated values as separate arguments. Syntax:

```
tagFunction`string ${expression} string`
```

Example:

```
function highlight(strings, ...values) {
    return strings.reduce((result, str, i) => {
      const val = values[i] !== undefined ? `<mark>${values[i]}</mark>` : "";
      return result + str + val;
    }, "");
  }

const name = "Alice";
  const score = 95;
  highlight`Student ${name} scored ${score}%`;
  // "Student <mark>Alice</mark> scored <mark>95</mark>%"
```

Real-world use cases:

```
- CSS-in-JS (styled-components): styled.div`color: red;`
  - SQL query building (prevent SQL injection by escaping values)
  - i18n (internationalization)
  - GraphQL queries: gql`query { users { id name } }`
```

Q77

What is Object.defineProperty()?

Object.defineProperty() lets you define or modify a property with fine-grained control over its behaviour using a property descriptor. Syntax:

```
Object.defineProperty(obj, propertyName, descriptor)
```

Descriptor options: value – the property's value

```
writable     – if false, value cannot be changed (default: false)
  enumerable   – if false, won't appear in for...in or Object.keys (default: false)
  configurable – if false, property cannot be deleted or redefined (default: false)
  get/set      – getter and setter functions (cannot combine with value/writable)
```

Example:

```
const person = {};
  Object.defineProperty(person, "name", {
```

value: "Alice", writable: false, enumerable: true, configurable: false

```
});
  person.name = "Bob"; // silently fails (or TypeError in strict mode)
  person.name;         // "Alice"

// Non-enumerable:
  Object.defineProperty(person, "secret", { value: "shhh", enumerable: false });
  Object.keys(person); // ["name"] (secret is hidden)
```

Q78

What is the difference between Object.freeze() and Object.seal()?

Object.freeze(obj):

```
- Cannot ADD new properties
```

- Cannot REMOVE existing properties - Cannot CHANGE existing property values - Properties become non-writable and non-configurable

```
- SHALLOW freeze (nested objects are not frozen)
```

```
const obj = Object.freeze({ a: 1, b: { c: 2 } });
  obj.a = 10;    // silently fails
  obj.d = 4;     // silently fails
  obj.b.c = 99;  // WORKS (shallow freeze doesn't affect nested objects)
```

Object.seal(obj):

```
- Cannot ADD new properties
```

- Cannot REMOVE existing properties

```
- CAN CHANGE values of existing properties (if writable)
```

```
const obj = Object.seal({ a: 1 });
  obj.a = 10;    // WORKS (can modify existing)
  obj.b = 2;     // silently fails (cannot add new)
  delete obj.a;  // silently fails (cannot delete)
```

Summary: - freeze: no add, no delete, no modify - seal: no add, no delete, YES modify

---

Q79

What is optional chaining (?.) and nullish coalescing (??)?

Optional Chaining (?.) – ES2020:

```
Allows safe access to deeply nested properties without checking each level for null/undefined. Returns und
```

```
const user = { profile: { address: { city: "Mumbai" } } };
  user?.profile?.address?.city;     // "Mumbai"
  user?.settings?.theme;            // undefined (no error)
  user?.getName?.();                // undefined if getName doesn't exist
  arr?.[0];                         // safe array access
```

Nullish Coalescing (??) – ES2020: Returns the right-hand side operand when the left-hand side is null OR undefined. Unlike ||, it does NOT treat 0, false, or "" as "no value".

```
null ?? "default"      // "default"
  undefined ?? "default" // "default"
  0 ?? "default"         // 0   (0 is a valid value!)
  "" ?? "default"        // ""  (empty string is valid)
  false ?? "default"     // false
```

Combining both:

```
const city = user?.profile?.address?.city ?? "Unknown";
```

Miscellaneous / Common Interview Questions

Q80

What is the difference between '==' with null/undefined?

null == undefined evaluates to true with loose equality. null and undefined are ONLY loosely equal to each other – they are not loosely equal to any other falsy value:

```
null == undefined  // true
  null == 0          // false
  null == ""         // false
  null == false      // false
  undefined == 0     // false
  undefined == ""    // false
```

Practical use:

```
// Check for both null and undefined:
  if (value == null) { /* value is null or undefined */ }
  // Equivalent to: value === null || value === undefined
```

Q81

Explain how 'typeof' works

typeof returns a string indicating the type of the operand:

```
typeof 42            // "number"
  typeof "hello"       // "string"
  typeof true          // "boolean"
  typeof undefined     // "undefined"
  typeof Symbol()      // "symbol"
  typeof 42n           // "bigint"
  typeof function(){}  // "function"
  typeof {}            // "object"
  typeof []            // "object"   (arrays are objects)
  typeof null          // "object"   (historical bug – null is NOT an object)
```

Checking for array: Array.isArray(val) Checking for null: val === null More specific type checking: Object.prototype.toString.call(val)

```
Object.prototype.toString.call([])   // "[object Array]"
  Object.prototype.toString.call(null) // "[object Null]"
  Object.prototype.toString.call(/re/) // "[object RegExp]"
```

Q82

What is the difference between function declaration and function expression?

Function Declaration:

```
function greet(name) {
    return "Hello " + name;
  }
  - Hoisted completely (can be called before the declaration)
```

- Always named

```
- Creates a named function in the current scope
```

Function Expression:

```
const greet = function(name) {
    return "Hello " + name;
  };
  - NOT hoisted (variable is hoisted but NOT the function value)
```

- Can be anonymous or named

```
- The name is scoped to the function itself (for recursion/debugging)
```

```
greet();  // Works with declaration
  // greet(); // TypeError with expression (greet is undefined when called before)
```

Named Function Expression:

```
const factorial = function fact(n) {
    return n <= 1 ? 1 : n * fact(n - 1); // 'fact' is only accessible inside
  };
```

Q83

What is the 'arguments' object?

The 'arguments' object is an array-like object accessible inside regular (non-arrow) functions that contains the values of the arguments passed to that function.

```
function sum() {
    let total = 0;
    for (let i = 0; i < arguments.length; i++) {
      total += arguments[i];
    }
    return total;
  }
  sum(1, 2, 3); // 6
```

Important notes:

```
- arguments is NOT a real Array (no map, filter, etc.)
  - Convert to array: Array.from(arguments) or [...arguments]
  - NOT available in arrow functions
```

Modern alternative – use rest parameters:

```
function sum(...nums) {
    return nums.reduce((acc, n) => acc + n, 0);
  }
  sum(1, 2, 3, 4); // 10
```

Q84

What is JSON and how do you work with it in JavaScript?

JSON (JavaScript Object Notation) is a lightweight data interchange format. It is text-based and easy for both humans and machines to read/write. JSON rules: - Keys must be double-quoted strings - Values can be: string, number, boolean, null, array, or object

```
- No functions, undefined, Dates (natively), or comments
```

JavaScript methods:

```
JSON.stringify(value, replacer, space):
```

- Converts a JavaScript value to a JSON string

```
- replacer: filter function or array of keys to include
  - space: indentation (number of spaces or string)
```

```
const obj = { name: "Alice", age: 30, active: true };
  JSON.stringify(obj); // '{"name":"Alice","age":30,"active":true}'
  JSON.stringify(obj, null, 2); // pretty-printed
```

```
JSON.parse(text, reviver):
```

- Converts a JSON string back to a JavaScript value

```
const parsed = JSON.parse('{"name":"Alice","age":30}');
  parsed.name; // "Alice"
```

Common error: JSON.parse throws SyntaxError on invalid JSON.

Q85

What is the event queue (task queue) in JavaScript?

The task queue (also called the callback queue or macrotask queue) holds callbacks from asynchronous Web API operations that are ready to be executed. How it works:

```
1. Asynchronous operation is handed to a Web API (e.g., setTimeout)
  2. Web API processes the operation (waits for the timer)
```

3. When ready, the callback is moved to the Task Queue 4. The event loop checks: if the call stack is empty, moves the first task from the queue to the stack 5. The callback executes on the call stack Two queues:

```
- Microtask Queue: Promise callbacks (higher priority – ALL drained before next macrotask)
  - Macrotask Queue: setTimeout, setInterval, I/O (lower priority – one per event loop tick)
```

This is why Promise callbacks ALWAYS run before setTimeout callbacks, even if both are ready:

```
setTimeout(() => console.log("timeout"), 0);
  Promise.resolve().then(() => console.log("promise"));
  // Output: "promise" then "timeout"
```

Q86

What are some common array and string manipulation interview problems?

Reverse a string:

```
"hello".split("").reverse().join(""); // "olleh"
```

Check palindrome:

```
const isPalindrome = s => s === s.split("").reverse().join("");
```

Find duplicates in array:

```
const dups = arr => arr.filter((item, i) => arr.indexOf(item) !== i);
  // or using Set:
  const dups = arr => [...new Set(arr.filter((v, i) => arr.indexOf(v) !== i))];
```

Flatten nested array:

```
arr.flat(Infinity);
```

Remove falsy values:

```
arr.filter(Boolean);
```

Count occurrences:

```
const count = arr.reduce((acc, val) => {
    acc[val] = (acc[val] || 0) + 1; return acc;
  }, {});
```

Find most frequent element:

```
const arr = [1,2,2,3,3,3];
  const freq = arr.reduce((a,v) => ({...a,[v]:(a[v]||0)+1}), {});
  const max = Object.keys(freq).reduce((a,b) => freq[a] > freq[b] ? a : b);
```

Chunk array:

```
const chunk = (arr, size) =>
    Array.from({ length: Math.ceil(arr.length / size) }, (_, i) =>
      arr.slice(i * size, i * size + size));
```

Security

Q87

What is Cross-Site Scripting (XSS) and how do you prevent it?

XSS is a security attack where an attacker injects malicious scripts into web pages viewed by other users. The injected script runs in the victim's browser with the permissions of the trusted site. Types:

```
- Stored XSS: Malicious script stored in the database (comments, user profiles)
```

- Reflected XSS: Script reflected off the server via URL parameters - DOM-based XSS: Attacker modifies the DOM environment Prevention:

```
1. Sanitize and escape user input before rendering (never use innerHTML with untrusted data)
```

2. Use textContent instead of innerHTML:

```
el.textContent = userInput; // safe
     el.innerHTML = userInput;   // DANGEROUS
```

```
3. Content Security Policy (CSP) headers – restrict which scripts can run
  4. Use HTTP-only cookies (prevent JS access to cookies)
```

5. Validate and sanitize on the server

```
6. Use trusted libraries/frameworks that escape by default (React, Vue escape JSX expressions)
```

Q88

What is CORS and how does it work?

CORS (Cross-Origin Resource Sharing) is a security mechanism that restricts web pages from making requests to a different domain, protocol, or port than the one that served the web page (same-origin policy). An origin is: scheme (https) + hostname (example.com) + port (443) CORS works via HTTP headers:

```
- Server sends Access-Control-Allow-Origin: https://mysite.com (or *)
```

- Browser compares the origin of the page with this header

```
- If not allowed, browser blocks the response (the request IS sent, but response is hidden)
```

Simple requests (GET, POST with basic content types) – browser adds Origin header automatically. Preflight requests (OPTIONS):

```
- For complex requests (PUT, DELETE, custom headers, JSON body)
```

- Browser sends OPTIONS request first to check if server allows it - Server must respond with appropriate CORS headers CORS headers: Access-Control-Allow-Origin Access-Control-Allow-Methods Access-Control-Allow-Headers Access-Control-Allow-Credentials Access-Control-Max-Age

---